🎯 GLA UNIVERSITY — DAY 2 POST-ASSESSMENT 50 MCQS (OFFICIAL INSTRUCTOR MASTER)

Traditional Machine Learning & Parallel Computing using Intel oneAPI (`GLA-HPC-BCSG701`)

Lead Trainer: Shivam Gupta | **Target Session:** Day 2 Conclusion (Evaluative Mastery Assessment)
Total Questions: 50 Multi-Choice Questions | **Target Pass Rate:** 85% – 95% Learning Gain
Evaluation Specifications: Official 7-Column University Examination Spec

---

📌 Pedagogical Architecture & Delta Evaluation Strategy

Administered at the conclusion of Day 2 (04:00 PM – 05:00 PM), this 50-question evaluative assessment proves learning ROI across:

- **Module 1 (Q01 – Q10):** Parallel Computing Principles & Scaling Mathematics (Amdahl, Gustafson, Superlinear scaling, Karp-Flatt, UMA vs NUMA)
- **Module 2 (Q11 – Q20):** OpenMP for Multithreading (Loop-carried dependencies, firstprivate, nowait, guided schedule, thread binding, collapse)
- **Module 3 (Q21 – Q30):** MPI for Distributed Systems (Comm split, non-blocking Isend/Irecv, Alltoall, derived vector types, binomial trees, RMA)
- **Module 4 (Q31 – Q40):** Data Decomposition & Synchronization (MESI cache protocol, 2D/3D halo depth, ThreadSanitizer, work-stealing, first-touch)
- **Module 5 (Q41 – Q50):** Intel AI Analytics Toolkit & oneDAL Acceleration (oneDAL architecture, patch_sklearn, Lloyd GEMM, Random Forest SIMD, GPU dpctl)

Question 01 [Module 1: Parallel Computing Principles & Scaling Mathematics | Medium | Mathematical Derivation]

Topic: Amdahl's Law Speedup on 64 Cores
Question: An image preprocessing pipeline has a parallel fraction of 95% ($P = 0.95$) and a serial fraction of 5% ($S = 0.05$). What is the theoretical speedup when scaled to 64 CPU cores according to Amdahl's Law?

- **a)** 64.0x
- **b)** 15.42x
- **c)** 20.0x
- **d)** 32.0x

> **TIP**
>
> **Correct Answer: [b]** — 15.42x
> **Technical Explanation:** Amdahl's Law: Speedup = 1 / [S + (P / N)] = 1 / [0.05 + (0.95 / 64)] = 1 / [0.05 + 0.01484] = 1 / 0.06484 ≈ 15.42x.

Question 02 [Module 1: Parallel Computing Principles & Scaling Mathematics | Hard | Mathematical Derivation]

Topic: Gustafson's Law Scaled Speedup
Question: Under Gustafson's Law for weak scaling, if the serial fraction of a scaled workload is 5% ($\alpha = 0.05$), what is the scaled speedup $S(P) = P - \alpha(P - 1)$ on 64 processors?

- **a)** 15.42x
- **b)** 60.85x
- **c)** 64.0x
- **d)** 3.2x

> **TIP**
>
> **Correct Answer: [b]** — 60.85x
> **Technical Explanation:** Gustafson's Law: $S(P) = P - \alpha(P - 1) = 64 - 0.05(64 - 1) = 64 - 0.05(63) = 64 - 3.15 = 60.85x$ scaled speedup.

Question 03 [Module 1: Parallel Computing Principles & Scaling Mathematics | Hard | Hardware Architecture]

Topic: Superlinear Speedup Physical Mechanism
Question: In empirical benchmarks, an algorithm running on 8 CPU cores achieves a speedup of 9.2x (greater than 8x). What is the primary physical hardware explanation for this 'Superlinear Speedup'?

- **a)** The CPU frequency spontaneously tripled due to quantum resonance
- **b)** Cache Aggregation: Partitioning the dataset across 8 cores reduced each core's working set size such that it fit entirely within fast L2/L3 on-chip caches, eliminating slow DDR main memory accesses
- **c)** The operating system disabled interrupt processing permanently
- **d)** The C++ compiler skipped executing half of the loop iterations

> **TIP**
>
> **Correct Answer: [b]** — Cache Aggregation: Partitioning the dataset across 8 cores reduced each core's working set size such that it fit entirely within fast L2/L3 on-chip caches, eliminating slow DDR main memory accesses
> **Technical Explanation:** Superlinear speedup ($S > N$) occurs when dividing the problem among N processors allows each processor's local working set to fit completely inside low-latency L2/L3 caches. On 1 core, the entire dataset exceeded cache size, causing frequent high-latency RAM stalls.

Question 04 [Module 1: Parallel Computing Principles & Scaling Mathematics | Hard | Performance Metrics]

Topic: Karp-Flatt Metric for Serial Overhead
Question: What does the Karp-Flatt metric $e = \frac{1/S(P) - 1/P}{1 - 1/P}$ measure when analyzing parallel scaling across increasing processor counts?

- **a)** The electrical power consumption of the CPU socket in Watts
- **b)** The experimentally observed serial fraction of the parallel application, revealing whether scaling degradation is caused by algorithmic serial bottlenecks or communication/synchronization overhead
- **c)** The percentage of network packet drop in InfiniBand fabrics
- **d)** The clock frequency jitter across NUMA nodes

> **TIP**
> **Correct Answer: [b]** — The experimentally observed serial fraction of the parallel application, revealing whether scaling degradation is caused by algorithmic serial bottlenecks or communication/synchronization overhead
> **Technical Explanation:** The Karp-Flatt metric calculates the effective serial fraction e from measured speedup $S(P)$. If e remains constant as P grows, the bottleneck is algorithmic serial code. If e increases with P, the bottleneck is parallel communication and synchronization overhead.

Question 05 [Module 1: Parallel Computing Principles & Scaling Mathematics | Medium | HPC Scaling]

Topic: Communication-to-Computation Ratio
Question: How does the Communication-to-Computation ratio (C/C) dictate the scalability of a distributed parallel algorithm as cluster node count increases?

- **a)** A high C/C ratio guarantees linear scaling to thousands of nodes
- **b)** As node count increases, if the communication overhead grows faster than local computation shrinks, the application encounters an efficiency cliff where adding more nodes slows down execution
- **c)** Communication overhead is always zero in MPI
- **d)** Computation time automatically increases when network switches are fast

> **TIP**
> **Correct Answer: [b]** — As node count increases, if the communication overhead grows faster than local computation shrinks, the application encounters an efficiency cliff where adding more nodes slows down execution
> **Technical Explanation:** For scalable parallelism, the computation per node must significantly outweigh communication overhead ($T_{comp} \gg T_{comm}$). As more nodes are added to a fixed-size problem, compute time per node decreases while collective communication overhead increases, eventually causing performance collapse.

Question 06 [Module 1: Parallel Computing Principles & Scaling Mathematics | Medium | Performance Analysis]

Topic: Parallel Overhead Breakdown
Question: Which of the following is NOT an inherent source of parallel execution overhead in multi-threaded/distributed systems?

- **a)** Thread creation, scheduling, and context-switching overhead
- **b)** Synchronization barriers and lock contention latency

- **c)** Inter-process network message transfer latency
- **d)** Vector SIMD single-instruction mathematical throughput

> **TIP**
> **Correct Answer: [d]** — Vector SIMD single-instruction mathematical throughput
> **Technical Explanation:** Vector SIMD execution is on-chip computational acceleration, not parallel overhead. The classic sources of parallel overhead are: (1) inter-process communication latency, (2) synchronization barriers/locks, (3) thread/process management overhead, and (4) load imbalance.

Question 07 [Module 1: Parallel Computing Principles & Scaling Mathematics | Medium | Memory Systems]

Topic: UMA vs NUMA Architecture
Question: What distinguishes Uniform Memory Access (UMA) from Non-Uniform Memory Access (NUMA) multi-socket systems?

- **a)** UMA systems have no RAM; NUMA systems have infinite RAM
- **b)** In UMA, memory access latency is identical for all CPUs to all memory locations; in NUMA, access latency depends on whether memory is attached to the local CPU socket or a remote CPU socket across the interconnect bus
- **c)** NUMA only works on 32-bit operating systems
- **d)** UMA is only used for GPUs

> **TIP**
> **Correct Answer: [b]** — In UMA, memory access latency is identical for all CPUs to all memory locations; in NUMA, access latency depends on whether memory is attached to the local CPU socket or a remote CPU socket across the interconnect bus
> **Technical Explanation:** In SMP/UMA, all processors share a centralized memory bus with uniform latency. In modern multi-socket NUMA (e.g., dual Xeon), each socket has dedicated local memory controllers. Accessing remote memory attached to another socket over UPI incurs significantly higher latency.

Question 08 [Module 1: Parallel Computing Principles & Scaling Mathematics | Hard | Parallel Scaling]

Topic: Isoefficiency Metric
Question: In parallel scalability theory, what does the 'Isoefficiency Function' quantify?

- **a)** The rate at which problem size W must grow with processor count P to maintain a constant parallel efficiency E
- **b)** The thermal efficiency of CPU fans
- **c)** The ratio of integer operations to floating-point operations
- **d)** The speed of the network router divided by RAM speed

> **TIP**
> **Correct Answer: [a]** — The rate at which problem size W must grow with processor count P to maintain a constant parallel efficiency E
> **Technical Explanation:** The isoefficiency metric determines how scalable an algorithm is: it mathematically expresses how rapidly the workload W must increase as a function of processor count P to hold efficiency constant. A smaller isoefficiency function ($O(P)$ vs $O(P^2)$) indicates superior scalability.

Question 09 [Module 1: Parallel Computing Principles & Scaling Mathematics | Medium | Hardware Bottlenecks]

Topic: Thread Concurrency vs Memory Bandwidth
Question: Why does running 64 threads on a 64-core Xeon server often produce zero speedup for a streaming memory operation like `y[i] = a * x[i] + b` (STREAM Triad)?

- **a)** Because OpenMP limits thread count to 8
- **b)** Because the workload is strictly memory-bandwidth bound, and a small number of cores (e.g., 8-12 cores) already completely saturate the physical DDR5 memory channels
- **c)** Because addition operations cannot be vectorized
- **d)** Because Linux CFS scheduler suspends threads that write memory

> **TIP**
> **Correct Answer: [b]** — Because the workload is strictly memory-bandwidth bound, and a small number of cores (e.g., 8-12 cores) already completely saturate the physical DDR5 memory channels
> **Technical Explanation:** The STREAM Triad has an arithmetic intensity of only 0.08 FLOPs/byte. A modern dual-socket Xeon saturates its total physical DDR5 memory bandwidth (e.g., ~600 GB/s) using just 8 to 16 cores. Adding more cores cannot increase memory bus bandwidth.

Question 10 [Module 1: Parallel Computing Principles & Scaling Mathematics | Medium | Performance Analysis]

Topic: Strong Scaling Efficiency Limit
Question: Under strong scaling, what ultimately causes the speedup curve to bend downwards (negative speedup) at very high core counts?

- **a)** The problem size per core becomes so small that inter-thread communication, barrier synchronization, and serial tail latency dominate total runtime
- **b)** The compiler optimization level degrades automatically
- **c)** The CPU enters sleep mode due to lack of instructions
- **d)** The hard disk fills up with compiler logs

> **TIP**
> **Correct Answer: [a]** — The problem size per core becomes so small that inter-thread communication, barrier synchronization, and serial tail latency dominate total runtime
> **Technical Explanation:** In strong scaling, as core count $N \to \infty$, the work allocated per core drops to $W/N \to 0$. Meanwhile, barrier overhead, thread creation, and inter-core cache-coherence traffic increase, causing total runtime to increase.

Question 11 [Module 2: OpenMP for Multithreading | Hard | Loop Parallelization]

Topic: Loop-Carried Dependencies in OpenMP
Question: Why will `#pragma omp parallel for` produce incorrect mathematical results when applied to: `for (int i=1; i<N; i++) fib[i] = fib[i-1] + fib[i-2];`?

- **a)** Because OpenMP does not support the Fibonacci sequence

- **b)** Because the loop contains a true Read-After-Write (RAW) loop-carried dependency where each iteration requires values computed in immediately preceding iterations
- **c)** Because array indexing must start from 0 in parallel loops
- **d)** Because `fib` is a reserved keyword in C++

> **TIP**
> **Correct Answer: [b]** — Because the loop contains a true Read-After-Write (RAW) loop-carried dependency where each iteration requires values computed in immediately preceding iterations
> **Technical Explanation:** Parallelizing a loop with loop-carried dependencies causes race conditions: thread A executing iteration 5 may execute before thread B finishes iteration 4, reading uninitialized or stale values. Such loops must be restructured or computed sequentially.

Question 12 [Module 2: OpenMP for Multithreading | Medium | Variable Scoping]

Topic: Data Scoping: `firstprivate` vs `private`
Question: What is the critical behavioral difference between `private(x)` and `firstprivate(x)` in an OpenMP parallel region?

- **a)** `firstprivate(x)` initializes each thread's private copy of `x` with the value `x` had immediately prior to entering the parallel construct; `private(x)` leaves the private copy uninitialized
- **b)** `private(x)` allows all threads to write to the same memory location
- **c)** `firstprivate(x)` only creates a private variable for Thread 0
- **d)** `firstprivate(x)` allocates `x` on the GPU

> **TIP**
> **Correct Answer: [a]** — `firstprivate(x)` initializes each thread's private copy of `x` with the value `x` had immediately prior to entering the parallel construct; `private(x)` leaves the private copy uninitialized
> **Technical Explanation:** `private(var)` creates an uninitialized local copy for each thread; reading it inside the parallel region before writing causes undefined behavior. `firstprivate(var)` creates a private copy initialized with the master thread's existing value.

Question 13 [Module 2: OpenMP for Multithreading | Medium | Synchronization Primitives]

Topic: OpenMP Synchronization: Barrier Directives
Question: What does `#pragma omp barrier` enforce within a parallel team of threads?

- **a)** It terminates all threads immediately
- **b)** It forces every thread in the team to halt at that execution point until all other threads in the team have arrived, ensuring memory consistency before proceeding
- **c)** It locks the motherboard memory bus for 1 second
- **d)** It prints the thread IDs to the console

> **TIP**
> **Correct Answer: [b]** — It forces every thread in the team to halt at that execution point until all other threads in the team have arrived, ensuring memory consistency before proceeding
> **Technical Explanation:** A barrier is an explicit synchronization point. No thread in the parallel team can proceed past `#pragma omp barrier` until all threads in that team have reached the barrier.

Question 14 [Module 2: OpenMP for Multithreading | Hard | Performance Tuning]

Topic: OpenMP `nowait` Clause
Question: When is it safe and advantageous to append the `nowait` clause to a work-sharing construct like `#pragma omp for nowait`?

- **a)** When the compiler gives an error during compilation
- **b)** When subsequent code does not depend on the results of the completed loop, allowing faster threads to proceed immediately to the next block without waiting at the implicit barrier
- **c)** When executing on a single-core CPU
- **d)** When all loop variables are declared global

> **TIP**
> **Correct Answer: [b]** — When subsequent code does not depend on the results of the completed loop, allowing faster threads to proceed immediately to the next block without waiting at the implicit barrier
> **Technical Explanation:** OpenMP `#pragma omp for` has an implicit synchronization barrier at its closing brace. Appending `nowait` removes this implicit barrier, enabling threads that finish their assigned chunk early to immediately continue execution, improving throughput.

Question 15 [Module 2: OpenMP for Multithreading | Medium | Work-Sharing Directives]

Topic: OpenMP `single` vs `master` Directives
Question: What is the difference between `#pragma omp master` and `#pragma omp single` in OpenMP?

- **a)** `master` is executed exclusively by thread 0 with NO implicit barrier; `single` is executed by whichever thread arrives first and includes an implicit barrier at the end (unless `nowait` is specified)
- **b)** `single` executes on 1 core; `master` executes on all cores
- **c)** `master` is deprecated in OpenMP 5.0 and replaced by `python`
- **d)** `single` only executes once across the entire operating system boot

> **TIP**
> **Correct Answer: [a]** — `master` is executed exclusively by thread 0 with NO implicit barrier; `single` is executed by whichever thread arrives first and includes an implicit barrier at the end (unless `nowait` is specified)
> **Technical Explanation:** `#pragma omp master` (deprecated in 5.1 in favor of `masked`) executes strictly on thread 0 without an implicit barrier. `#pragma omp single` executes on whichever thread reaches it first, with an implicit barrier that all other threads wait at.

Question 16 [Module 2: OpenMP for Multithreading | Hard | Loop Scheduling]

Topic: OpenMP `schedule(guided, chunk)` Mechanics
Question: How does OpenMP `schedule(guided, chunk)` dynamically adjust chunk sizes across loop iterations?

- **a)** It randomly picks chunk sizes using a random number generator
- **b)** It starts by assigning large chunks of iterations to minimize scheduling overhead, and exponentially decreases chunk size down to `chunk` as remaining iterations diminish to balance the tail workload
- **c)** It divides all iterations equally among threads once at startup
- **d)** It assigns 1 iteration per thread strictly throughout the loop

> **TIP**
> **Correct Answer: [b]** — It starts by assigning large chunks of iterations to minimize scheduling overhead, and exponentially decreases chunk size down to `chunk` as remaining iterations diminish to balance the tail workload
> **Technical Explanation:** `guided` scheduling is an adaptive compromise between static and dynamic: chunk size is proportional to the number of unassigned iterations divided by thread count. As iterations decrease, chunk size shrinks down to a minimum of `chunk`, ensuring low scheduling overhead initially and fine balance at the end.

Question 17 [Module 2: OpenMP for Multithreading | Hard | Thread Affinity & NUMA]

Topic: OpenMP Thread Affinity: `OMP_PROC_BIND`
Question: What does setting `export OMP_PROC_BIND=close` versus `export OMP_PROC_BIND=spread` instruct the OpenMP runtime when placing threads across CPU sockets?

- **a)** `close` places threads consecutively on cores close to the master thread (sharing L3 cache); `spread` distributes threads evenly across separate sockets/NUMA nodes (maximizing memory bandwidth)
- **b)** `close` terminates threads when idle; `spread` duplicates threads
- **c)** `spread` spreads data across the network via Wi-Fi
- **d)** `close` limits execution to 1 thread

> **TIP**
> **Correct Answer: [a]** — `close` places threads consecutively on cores close to the master thread (sharing L3 cache); `spread` distributes threads evenly across separate sockets/NUMA nodes (maximizing memory bandwidth)
> **Technical Explanation:** `OMP_PROC_BIND=close` packs threads onto adjacent cores near the parent, maximizing shared cache locality. `OMP_PROC_BIND=spread` distributes threads as far apart as possible across separate NUMA sockets, maximizing aggregate memory bandwidth.

Question 18 [Module 2: OpenMP for Multithreading | Medium | Loop Parallelization]

Topic: OpenMP Collapse Clause
Question: What is the benefit of adding `collapse(2)` to `#pragma omp parallel for` on nested loops: `for (int i=0; i<N; i++) for (int j=0; j<M; j++)`?

- **a)** It deletes the outer loop entirely
- **b)** It flattens the two nested loops into a single linear iteration space of size $N \times M$, creating a much larger pool of parallel work items to distribute across large core counts
- **c)** It reduces memory usage by 50%

- **d)** It forces both loops to execute backwards

> **TIP**
> **Correct Answer: [b]** — It flattens the two nested loops into a single linear iteration space of size $N \times M$, creating a much larger pool of parallel work items to distribute across large core counts
> **Technical Explanation:** Without `collapse`, only the outermost loop (N iterations) is parallelized; if N is small (e.g., $N = 4$ on a 64-core system), 60 cores sit idle! `collapse(2)` combines $N \times M$ iterations into one iteration space, feeding all available threads.

Question 19 [Module 2: OpenMP for Multithreading | Hard | Task Parallelism]

Topic: OpenMP Tasking Model (`#pragma omp task`)
Question: Why is the OpenMP Tasking construct (`#pragma omp task`) ideal for parallelizing recursive algorithms (e.g., Quicksort or tree traversal) compared to standard `#pragma omp for`?

- **a)** Because `#pragma omp for` can only run on arrays with prime lengths
- **b)** Because tasks can be spawned dynamically and placed into a runtime work pool executed asynchronously by any available thread, enabling irregular and recursive parallelism
- **c)** Because tasks execute in the kernel space without memory stacks
- **d)** Because tasks disable compiler warnings

> **TIP**
> **Correct Answer: [b]** — Because tasks can be spawned dynamically and placed into a runtime work pool executed asynchronously by any available thread, enabling irregular and recursive parallelism
> **Technical Explanation:** Traditional `#pragma omp for` requires flat, predetermined canonical loop bounds known at loop entry. Recursive tree traversals and divide-and-conquer algorithms generate work dynamically at runtime; `#pragma omp task` packages these dynamic work units for asynchronous execution by idle threads.

Question 20 [Module 2: OpenMP for Multithreading | Medium | Defensive Parallel Programming]

Topic: OpenMP `default(none)` Best Practice
Question: Why do senior HPC engineers enforce `#pragma omp parallel default(none)` in all production C/C++ OpenMP codebases?

- **a)** To disable all compiler optimizations
- **b)** It forces the programmer to explicitly declare the data-sharing attribute (`shared`, `private`, etc.) of every single variable used in the construct, catching accidental race conditions at compile time
- **c)** To prevent the code from compiling on AMD processors
- **d)** To force OpenMP to run sequentially

> **TIP**
> **Correct Answer: [b]** — It forces the programmer to explicitly declare the data-sharing attribute (`shared`, `private`, etc.) of every single variable used in the construct, catching accidental race conditions at compile time
> **Technical Explanation:** By default, variables declared outside an OpenMP construct become `shared`. If a programmer forgets to declare a local temporary variable as `private`, all threads silently overwrite each other's data (data race). `default(none)` makes the compiler emit an error for any undeclared variable.

Question 21 [Module 3: MPI for Distributed Systems | Hard | Advanced MPI]

Topic: MPI Communicator Splitting (`MPI_Comm_split`)
Question: What is the primary function of `MPI_Comm_split(comm, color, key, &newcomm)` in high-performance computing clusters?

- **a)** To divide the cluster electrical power among nodes
- **b)** To partition an existing communicator into disjoint sub-communicators based on a `color` parameter, enabling localized group collective communication (e.g., node-local communicators)
- **c)** To terminate half of the MPI processes
- **d)** To split a single file into two pieces on the hard drive

> **TIP**
> **Correct Answer: [b]** — To partition an existing communicator into disjoint sub-communicators based on a `color` parameter, enabling localized group collective communication (e.g., node-local communicators)
> **Technical Explanation:** `MPI_Comm_split` partitions the processes of `comm` into disjoint groups where all processes with the same `color` form a new communicator `newcomm` . This is essential for hierarchical communication (e.g., intra-node vs inter-node communication).

Question 22 [Module 3: MPI for Distributed Systems | Hard | MPI Communication]

Topic: Non-Blocking Communication (`MPI_Isend` / `MPI_Irecv`)
Question: What is the key advantage of non-blocking communication (`MPI_Isend` , `MPI_Irecv`) paired with `MPI_Wait()` in high-concurrency scientific simulations?

- **a)** It doubles the network wire speed
- **b)** It initiates message transfer asynchronously and returns immediately, allowing the CPU to compute interior domain cells while network hardware transfers boundary data in parallel (computation-communication overlap)
- **c)** It eliminates the need for network switches
- **d)** It converts MPI messages into encrypted emails

> **TIP**
> **Correct Answer: [b]** — It initiates message transfer asynchronously and returns immediately, allowing the CPU to compute interior domain cells while network hardware transfers boundary data in parallel (computation-communication overlap)
> **Technical Explanation:** Non-blocking calls decouple communication initiation from completion. While the network interface card (NIC/HCA) handles RDMA transfers in the background, the CPU continues computing interior grid points, hiding communication latency entirely.

Question 23 [Module 3: MPI for Distributed Systems | Hard | MPI Protocols]

Topic: MPI Communication Modes: Standard, Synchronous, Buffered
Question: What distinguishes Synchronous Send (`MPI_Ssend`) from Standard Send (`MPI_Send`) in the MPI standard?

- **a)** `MPI_Ssend` is 100 times faster than `MPI_Send`
- **b)** `MPI_Ssend` guarantees that the send operation will NOT complete until the matching receive has been posted and has started receiving data (rendezvous handshake); standard `MPI_Send` may buffer and return early
- **c)** `MPI_Ssend` only operates on single-byte characters

- **d)** `MPI_Ssend` sends messages through hard disk files

> **TIP**
> **Correct Answer: [b]** — `MPI_Ssend` guarantees that the send operation will NOT complete until the matching receive has been posted and has started receiving data (rendezvous handshake); standard `MPI_Send` may buffer and return early
> **Technical Explanation:** `MPI_Send` protocol depends on message size: small messages use eager protocol (buffered on destination, returns early); large messages use rendezvous. `MPI_Ssend` unconditionally enforces a rendezvous handshake, returning only after the receiver begins receiving.

Question 24 [Module 3: MPI for Distributed Systems | Hard | MPI Collectives]

Topic: Collective All-to-All Communication (`MPI_Alltoall`)
Question: In an MPI cluster of P ranks, what data exchange pattern is performed by `MPI_Alltoall` ?

- **a)** Rank 0 sends data to Rank 1 only
- **b)** Every rank sends a distinct chunk of data to every other rank in the communicator (a complete distributed matrix transpose / all-to-all personalized exchange)
- **c)** Every rank sends data to the internet
- **d)** All ranks delete their local data arrays

> **TIP**
> **Correct Answer: [b]** — Every rank sends a distinct chunk of data to every other rank in the communicator (a complete distributed matrix transpose / all-to-all personalized exchange)
> **Technical Explanation:** `MPI_Alltoall` is the complete personalized cross-bar exchange: each process sends P distinct buffers, one to each of the P processes, and receives P distinct buffers from each process. It is the fundamental communication kernel of distributed 2D/3D Fast Fourier Transforms (FFTs).

Question 25 [Module 3: MPI for Distributed Systems | Medium | Defensive MPI]

Topic: Deadlock Avoidance with `MPI_Sendrecv`
Question: Why does using `MPI_Sendrecv` prevent the classic deadlock that occurs when adjacent processes attempt simultaneous blocking `MPI_Send` and `MPI_Recv` operations?

- **a)** It disables network error checking
- **b)** It executes the send and receive operations atomically under an internally managed non-blocking protocol, guaranteeing that communication proceeds without circular buffer dependency deadlocks
- **c)** It forces one process to terminate itself
- **d)** It writes both messages to a shared SQLite database

> **TIP**
> **Correct Answer: [b]** — It executes the send and receive operations atomically under an internally managed non-blocking protocol, guaranteeing that communication proceeds without circular buffer dependency deadlocks
> **Technical Explanation:** `MPI_Sendrecv` coordinates sending and receiving in a single atomic call. The MPI runtime handles the underlying buffers so that processes swapping boundary data do not dead-end waiting for the other to post a receive first.

Question 26 [Module 3: MPI for Distributed Systems | Hard | MPI Memory Optimization]

Topic: MPI Derived Datatypes (`MPI_Type_vector`)
Question: Why would an HPC developer create an `MPI_Type_vector` datatype instead of copying matrix elements into a temporary contiguous buffer before calling `MPI_Send`?

- **a)** To allow sending data to non-MPI programs
- **b)** To enable zero-copy transfers of non-contiguous strided memory blocks (such as a matrix column in row-major layout) directly by the network hardware/driver, eliminating memory packing/unpacking overhead
- **c)** Because MPI does not support loops
- **d)** To encrypt data across sockets

> **TIP**
> **Correct Answer: [b]** — To enable zero-copy transfers of non-contiguous strided memory blocks (such as a matrix column in row-major layout) directly by the network hardware/driver, eliminating memory packing/unpacking overhead
> **Technical Explanation:** In row-major matrix layout, columns are strided (non-contiguous in RAM). Manually packing them into temporary arrays consumes memory and CPU cycles. `MPI_Type_vector` defines a strided datatype that the network HCA can gather directly via scatter-gather DMA.

Question 27 [Module 3: MPI for Distributed Systems | Hard | Algorithmic Analysis]

Topic: Collective Tree Algorithmic Complexity
Question: On a cluster with P nodes, what is the theoretical communication time complexity of an optimized `MPI_Bcast` implemented via a binomial spanning tree compared to a naive root loop?

- **a)** Binomial tree takes $O(\log_2 P)$ steps; naive loop takes $O(P)$ steps
- **b)** Binomial tree takes $O(P^2)$ steps; naive loop takes $O(1)$ steps
- **c)** Both take $O(P)$ steps
- **d)** Binomial tree takes infinite steps

> **TIP**
> **Correct Answer: [a]** — Binomial tree takes $O(\log_2 P)$ steps; naive loop takes $O(P)$ steps
> **Technical Explanation:** A naive root loop sends $P - 1$ messages sequentially ($O(P)$ communication steps). A binomial tree doubles the number of informed processes at each step (Rank 0 informs 1; 0 and 1 inform 2 and 3; etc.), completing broadcast in $\lceil \log_2 P \rceil$ steps.

Question 28 [Module 3: MPI for Distributed Systems | Medium | MPI Safety]

Topic: MPI Wildcards and Out-of-Order Hazards
Question: What potential hazard arises when using `MPI_ANY_SOURCE` and `MPI_ANY_TAG` wildcards in high-throughput MPI message reception loops?

- **a)** The computer loses internet connection
- **b)** Non-deterministic message matching: messages from different sender ranks may arrive out of order, leading to non-reproducible race conditions or incorrect data association if matching logic is flawed
- **c)** The MPI library crashes with a syntax error
- **d)** The CPU frequency drops to 1 GHz

> **TIP**
>
> **Correct Answer: [b]** — Non-deterministic message matching: messages from different sender ranks may arrive out of order, leading to non-reproducible race conditions or incorrect data association if matching logic is flawed
> **Technical Explanation:** Using `MPI_ANY_SOURCE` receives whichever message arrives first. Because network delays fluctuate, message processing order becomes non-deterministic. Without careful tag and sequence tracking, simulations can process data out-of-order, producing corrupted results.

Question 29 [Module 3: MPI for Distributed Systems | Hard | MPI Topologies]

Topic: MPI Process Topologies (Cartesian Grids)
Question: What is the primary benefit of creating a virtual Cartesian grid topology via `MPI_Cart_create` in 2D/3D numerical physics simulations?

- **a)** It enables 3D graphics rendering on the cluster display
- **b)** It maps 1D linear MPI ranks onto logical multi-dimensional coordinates and provides `MPI_Cart_shift` to automatically identify neighboring rank IDs in North, South, East, and West directions
- **c)** It forces all compute nodes to be placed in the same room
- **d)** It doubles the RAM available to each process

> **TIP**
>
> **Correct Answer: [b]** — It maps 1D linear MPI ranks onto logical multi-dimensional coordinates and provides `MPI_Cart_shift` to automatically identify neighboring rank IDs in North, South, East, and West directions
> **Technical Explanation:** `MPI_Cart_create` structures linear ranks into an N-dimensional grid. Functions like `MPI_Cart_shift` then calculate neighbor ranks (e.g., $+1$ or -1 along an axis with periodic boundary conditions) without requiring manual modulo arithmetic.

Question 30 [Module 3: MPI for Distributed Systems | Hard | Advanced MPI]

Topic: One-Sided Communication (Remote Memory Access - RMA)
Question: What defines MPI-2/MPI-3 One-Sided Communication (Remote Memory Access / RMA) through `MPI_Put` and `MPI_Get`?

- **a)** Only one computer is turned on during execution
- **b)** A process can directly read from or write to the memory window of a remote process without requiring the remote process to explicitly participate in matching receive or send calls
- **c)** Messages can only be sent once per day
- **d)** RMA only works over serial cables

> **TIP**
>
> **Correct Answer: [b]** — A process can directly read from or write to the memory window of a remote process without requiring the remote process to explicitly participate in matching receive or send calls
> **Technical Explanation:** In two-sided MPI, every send requires an explicit matching receive on the remote target. In RMA (one-sided), an origin process defines an `MPI_Win` (window) and executes `MPI_Put` or `MPI_Get` directly into remote memory using RDMA hardware without remote CPU intervention.

Question 31 [Module 4: Data Decomposition & Synchronization | Hard | Hardware Physics]

Topic: MESI Cache Coherence Protocol States
Question: In the MESI cache coherence protocol, what happens when Core 1 writes to a 64-byte cache line currently held in the 'Shared' (S) state by Core 0 and Core 1?

- **a)** Core 1 sends an Invalidating Bus Transaction; Core 0 transitions its copy to 'Invalid' (I), while Core 1 updates the data and transitions its line to 'Modified' (M)
- **b)** Both cores freeze execution and reboot
- **c)** Core 0 overwrites Core 1's write immediately
- **d)** The cache line is permanently deleted from RAM

> **TIP**
> **Correct Answer: [a]** — Core 1 sends an Invalidating Bus Transaction; Core 0 transitions its copy to 'Invalid' (I), while Core 1 updates the data and transitions its line to 'Modified' (M)
> **Technical Explanation:** In the MESI protocol (Modified, Exclusive, Shared, Invalid): writing to a line in Shared (S) state requires invalidating all other cached copies. An invalidation signal is broadcast across the interconnect; other cores mark their line 'I', and the writing core transitions the line to 'M'.

Question 32 [Module 4: Data Decomposition & Synchronization | Hard | Performance Engineering]

Topic: False Sharing Mitigation via Cache Alignment
Question: How can a C++ developer permanently eliminate False Sharing when declaring per-thread state structures in OpenMP?

- **a)** By declaring all variables as `static`
- **b)** By using the C++11 `alignas(64)` specifier (or manual byte padding) to ensure each thread's independent structure starts on its own unique 64-byte cache line boundary
- **c)** By turning off CPU turbo boost in the BIOS
- **d)** By using 32-bit integers instead of 64-bit integers

> **TIP**
> **Correct Answer: [b]** — By using the C++11 `alignas(64)` specifier (or manual byte padding) to ensure each thread's independent structure starts on its own unique 64-byte cache line boundary
> **Technical Explanation:** False sharing occurs when independent variables share the same 64-byte hardware cache line. Aligning structures with `alignas(64)` forces the memory allocator to place each structure on a dedicated cache line, eliminating cross-core cache invalidation.

Question 33 [Module 4: Data Decomposition & Synchronization | Hard | Domain Decomposition]

Topic: Halo Cell Communication Depth
Question: In a distributed PDE solver using a 4th-order spatial central difference stencil that accesses 2 neighbors in each direction ($i-2, i-1, i, i+1, i+2$), what is the minimum required Halo (Ghost) cell layer thickness?

- **a)** 1 layer of cells
- **b)** 2 layers of cells
- **c)** 4 layers of cells
- **d)** 8 layers of cells

> **TIP**
> **Correct Answer: [b]** — 2 layers of cells
> **Technical Explanation:** Because the 4th-order stencil accesses neighbors up to distance 2 ($i \pm 2$), the boundary points inside the local domain require access to 2 exterior points belonging to adjacent subdomains. Thus, the ghost border must be at least 2 layers thick.

Question 34 [Module 4: Data Decomposition & Synchronization | Hard | Domain Decomposition]

Topic: 1D vs 2D vs 3D Domain Decomposition Scaling
Question: For a 3D grid simulation of size $N \times N \times N$ partitioned among P cluster processors, how does the communication surface area scale under 3D cube decomposition compared to 1D slab decomposition?

- **a)** 3D cube decomposition exchanges $O(N^2/P^{2/3})$ elements per node, which is dramatically smaller than 1D slab decomposition exchanging $O(N^2)$ elements per node at large P
- **b)** 1D slab decomposition has zero communication
- **c)** 3D decomposition requires 3 times more communication than 1D
- **d)** Both decompositions have identical communication volumes

> **TIP**
> **Correct Answer: [a]** — 3D cube decomposition exchanges $O(N^2/P^{2/3})$ elements per node, which is dramatically smaller than 1D slab decomposition exchanging $O(N^2)$ elements per node at large P
> **Technical Explanation:** In 1D slab decomposition, each partition has surface area $2N^2$ regardless of P. In 3D cubic decomposition, each subcube has edge length $N/P^{1/3}$, yielding a boundary surface area of $6(N/P^{1/3})^2 = 6N^2/P^{2/3}$. As P scales, 3D decomposition minimizes communication overhead.

Question 35 [Module 4: Data Decomposition & Synchronization | Medium | Debugging & Verification]

Topic: ThreadSanitizer & Race Detection
Question: Which compiler instrumentation flag can be passed to `gcc`, `clang`, or `icpx` to detect data races at runtime in multi-threaded C/C++ applications?

- **a)** `-O3`
- **b)** `-fsanitize=thread`
- **c)** `-fno-exceptions`
- **d)** `-Wall`

> **TIP**
> **Correct Answer: [b]** — `-fsanitize=thread`
> **Technical Explanation:** ThreadSanitizer (TSan, enabled via `-fsanitize=thread -g`) instruments memory accesses and synchronization primitives at compile time to detect data races, deadlocks, and use-after-free conditions dynamically at runtime.

Question 36 [Module 4: Data Decomposition & Synchronization | Hard | Load Balancing]

Topic: Dynamic Load Balancing via Work Stealing
Question: How does the 'Work Stealing' scheduling algorithm handle load imbalance in parallel task runtimes (e.g., Cilk,

Intel TBB)?

- **a)** It forces all processors to execute identical numbers of instructions
- **b)** Each processor maintains a local double-ended queue (deque) of tasks; when a processor runs out of work, it attempts to steal tasks from the tail of another processor's deque
- **c)** It terminates slow tasks and restarts them on the master thread
- **d)** It pauses all processors until the slowest processor finishes

> **TIP**
> **Correct Answer: [b]** — Each processor maintains a local double-ended queue (deque) of tasks; when a processor runs out of work, it attempts to steal tasks from the tail of another processor's deque
> **Technical Explanation:** In work-stealing, busy processors push/pop tasks from the front of their local deque (cache-friendly LIFO). When a worker becomes idle, it steals a task from the back of a randomly chosen victim's deque (FIFO), achieving provably optimal load balance with low contention.

Question 37 [Module 4: Data Decomposition & Synchronization | Hard | Lock-Free Programming]

Topic: Lock-Free Synchronization & CAS
Question: What fundamental CPU atomic instruction forms the basis of lock-free concurrent queues and stacks on modern x86_64 and ARM processors?

- **a)** Bitwise Shift Left (`SHL`)
- **b)** Compare-And-Swap (`CMPXCHG` on x86, `CAS` or `LDREX/STREX` on ARM)
- **c)** Floating-point Square Root (`FSQRT`)
- **d)** No-Operation (`NOP`)

> **TIP**
> **Correct Answer: [b]** — Compare-And-Swap (`CMPXCHG` on x86, `CAS` or `LDREX/STREX` on ARM)
> **Technical Explanation:** Compare-And-Swap (CAS) atomically compares the value in a memory location with an expected value, and if equal, updates it to a new value in a single uninterruptible hardware transaction, enabling lock-free data structures without OS mutexes.

Question 38 [Module 4: Data Decomposition & Synchronization | Hard | Memory Systems]

Topic: First-Touch Page Allocation on Linux NUMA
Question: When `malloc()` allocates a 50 GB array on a dual-socket Linux NUMA server, which physical socket's RAM controller is the memory allocated from?

- **a)** Socket 0 strictly, regardless of which thread touches it
- **b)** Neither socket at `malloc()` time; physical memory pages are only mapped when a thread FIRST writes to a page, allocating the page from the local NUMA memory node of the touching thread (First-Touch Policy)
- **c)** Memory is evenly distributed across both sockets at `malloc()` time
- **d)** Memory is allocated in the CPU L1 cache

> **TIP**
> **Correct Answer: [b]** — Neither socket at `malloc()` time; physical memory pages are only mapped when a thread FIRST writes to a page, allocating the page from the local NUMA memory node of the touching thread (First-Touch Policy)
> **Technical Explanation:** Linux uses lazy virtual memory allocation. Calling `malloc()` merely reserves virtual address space. Physical memory frames are assigned on the first page fault (when a thread first writes to the page), bound to the NUMA node of the executing thread.

Question 39 [Module 4: Data Decomposition & Synchronization | Hard | Lock-Free Concurrency]

Topic: ABA Problem in Lock-Free Concurrency
Question: What is the 'ABA Problem' in concurrent lock-free data structures using Compare-And-Swap (CAS)?

- **a)** An error when compiling code on ARM architecture
- **b)** A hazard where a memory location holds value A, changes to B, and changes back to A; a CAS operation succeeds assuming nothing changed, while intermediate pointer structures have been invalidated or reallocated
- **c)** When three threads run in alphabetical order
- **d)** A failure in the CPU floating-point unit

> **TIP**
> **Correct Answer: [b]** — A hazard where a memory location holds value A, changes to B, and changes back to A; a CAS operation succeeds assuming nothing changed, while intermediate pointer structures have been invalidated or reallocated
> **Technical Explanation:** The ABA problem occurs when thread 1 reads value A, thread 2 alters A to B and back to A, and thread 1 performs CAS expecting state to be unchanged. While the raw value matches A, underlying references or nodes may have been freed and recycled. It is solved using tagged pointers or Hazard Pointers.

Question 40 [Module 4: Data Decomposition & Synchronization | Hard | Hardware Consistency]

Topic: Memory Order Consistency Models
Question: Which memory consistency model is enforced by standard x86_64 hardware architecture for memory loads and stores?

- **a)** Total Store Order (TSO), where stores are not reordered with other stores, and loads are not reordered with other loads, but a store followed by a load may be reordered
- **b)** Strict Sequential Consistency for all operations with zero buffering
- **c)** Completely Relaxed Memory Model (ARM Weak Ordering)
- **d)** Random instruction reordering without store buffers

> **TIP**
> **Correct Answer: [a]** — Total Store Order (TSO), where stores are not reordered with other stores, and loads are not reordered with other loads, but a store followed by a load may be reordered
> **Technical Explanation:** x86 implements Total Store Order (TSO): stores to memory occur in program order, and loads occur in program order. The only reordering permitted is that a processor may read a memory location before a prior store by the same processor becomes globally visible (Store-Load reordering via store buffers).

Question 41 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | oneAPI Architecture]

Topic: Intel oneDAL Computational Engine
Question: What is the foundational architectural advantage of Intel oneDAL over generic open-source Scikit-Learn backends?

- **a)** oneDAL converts Python scripts into JavaScript
- **b)** oneDAL implements classical algorithms in optimized C++ utilizing multi-threading, CPU cache blocking, vectorization (AVX-512), and hardware matrix engines (Intel AMX) on modern Xeon processors
- **c)** oneDAL removes all math operations to run faster
- **d)** oneDAL requires zero electricity to run

> **TIP**
> **Correct Answer: [b]** — oneDAL implements classical algorithms in optimized C++ utilizing multi-threading, CPU cache blocking, vectorization (AVX-512), and hardware matrix engines (Intel AMX) on modern Xeon processors
> **Technical Explanation:** Standard Scikit-learn algorithms are often implemented in Python or generic Cython with unvectorized scalar loops. oneDAL re-engineers algorithms from the ground up to exploit Intel Xeon architecture: AVX-512 FMA vector registers, AMX tile registers, multi-socket NUMA awareness, and optimal L1/L2 cache blocking.

Question 42 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | Applied ML Acceleration]

Topic: Drop-in Patching Granularity in `sklearnex`
Question: If a developer wants to patch ONLY the K-Means and Support Vector Classifier (SVC) algorithms while keeping all other Scikit-Learn estimators unpatched, how can this be achieved?

- **a)** `patch_sklearn(['kmeans', 'svc'])`
- **b)** `patch_sklearn(name='kmeans', name2='svc')`
- **c)** `patch_sklearn(global=False)`
- **d)** Selective patching is impossible; `patch_sklearn()` always patches everything

> **TIP**
> **Correct Answer: [a]** — `patch_sklearn(['kmeans', 'svc'])`
> **Technical Explanation:** `sklearnex.patch_sklearn()` accepts an optional list parameter specifying exact algorithm names to accelerate: `patch_sklearn(['kmeans', 'svc'])`. This allows targeted acceleration while leaving other estimators untouched.

Question 43 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Hard | Algorithmic Acceleration]

Topic: oneDAL K-Means Algorithmic Acceleration
Question: How does Intel oneDAL optimize the Lloyd's algorithm iteration step in K-Means clustering for large datasets (e.g., 500,000 samples $\times$ 50 features)?

- **a)** By using the Triangle Inequality to filter out centroid distance calculations that cannot possibly change cluster assignment, and computing batch distances using GEMM matrix multiplication

- **b)** By assigning every point to cluster 0
- **c)** By running K-Means on only 10 data points
- **d)** By converting all numbers to 8-bit integers with loss of precision

> **TIP**
> **Correct Answer: [a]** — By using the Triangle Inequality to filter out centroid distance calculations that cannot possibly change cluster assignment, and computing batch distances using GEMM matrix multiplication
> **Technical Explanation:** oneDAL formulates distance computation $||x_i - c_j||^2 = ||x_i||^2 + ||c_j||^2 - 2x_i^T c_j$ as a BLAS Level 3 GEMM matrix multiplication, which achieves peak GFLOPS on AVX-512/AMX units. Furthermore, it leverages geometric triangle inequalities to skip unnecessary distance evaluations.

Question 44 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Hard | Algorithmic Acceleration]

Topic: oneDAL Random Forest Vectorized Tree Traversal
Question: What algorithmic innovation does oneDAL implement to achieve up to 20x faster inference for Random Forest classification?

- **a)** It deletes all decision trees except one
- **b)** It utilizes vectorized tree traversal, evaluating multiple data samples or multiple tree nodes simultaneously using SIMD vector registers rather than serial scalar pointer chasing
- **c)** It converts decision trees into neural networks
- **d)** It runs predictions on the hard drive

> **TIP**
> **Correct Answer: [b]** — It utilizes vectorized tree traversal, evaluating multiple data samples or multiple tree nodes simultaneously using SIMD vector registers rather than serial scalar pointer chasing
> **Technical Explanation:** Classical decision tree traversal evaluates one sample down one tree node at a time (branchy pointer chasing with poor cache locality and branch mispredictions). oneDAL transforms tree traversal into vectorized bitwise operations, evaluating batches of samples across multiple tree levels in SIMD registers.

Question 45 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | Intel AI Analytics Toolkit]

Topic: Intel-Optimized NumPy BLAS Integration
Question: When executing `np.dot(A, B)` on large matrices in an environment with the Intel AI Analytics Toolkit, why is performance substantially faster than default pip NumPy?

- **a)** Because Intel NumPy rounds numbers to the nearest integer
- **b)** Because it links directly against Intel oneMKL (Math Kernel Library), which dynamically dispatches multi-threaded AVX-512 and AMX assembly routines optimized for Xeon cache hierarchies
- **c)** Because it skips calculating the off-diagonal elements
- **d)** Because it runs in the Linux kernel space

> **TIP**
>
> **Correct Answer: [b]** — Because it links directly against Intel oneMKL (Math Kernel Library), which dynamically dispatches multi-threaded AVX-512 and AMX assembly routines optimized for Xeon cache hierarchies
> **Technical Explanation:** Default pip NumPy often links to reference OpenBLAS or Netlib BLAS compiled for baseline x86-64. Intel-optimized NumPy links to Intel oneMKL, which features architecture-specific kernels tuned for Intel Xeon cache line boundaries and vector widths.

Question 46 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | Intel AI Analytics Toolkit]

Topic: Modin Multi-Core Dataframe Execution
Question: What code modification is required in an existing Python script to accelerate Pandas data processing across all 64 cores of an HPC node using Modin?

- **a)** Rewrite all dataframe operations using SQL queries
- **b)** Replace `import pandas as pd` with `import modin.pandas as pd`
- **c)** Add `with multiprocessing.Pool(64):` around every line of Pandas code
- **d)** Convert all dataframes into CSV files on disk

> **TIP**
>
> **Correct Answer: [b]** — Replace `import pandas as pd` with `import modin.pandas as pd`
> **Technical Explanation:** Modin is a 100% API-compatible drop-in replacement for Pandas. Changing `import pandas as pd` to `import modin.pandas as pd` automatically activates its distributed Ray/Dask execution engine, partitioning dataframes across all available CPU cores.

Question 47 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Hard | Algorithmic Acceleration]

Topic: Support Vector Machine (SVC) Scaling with oneDAL
Question: Why does training a non-linear SVM (RBF kernel) on large datasets scale poorly in standard Scikit-Learn ($O(N^2)$ to $O(N^3)$), and how does oneDAL mitigate this bottleneck?

- **a)** Standard Scikit-learn cannot compute exponential functions; oneDAL approximates all kernels as linear
- **b)** Kernel SVM requires computing an $N \times N$ dense kernel matrix; oneDAL uses efficient block-kernel caching, vectorized matrix evaluations, and parallelized working set selection in the SMO solver
- **c)** oneDAL limits SVM training to a maximum of 100 samples
- **d)** oneDAL runs SVM on the cluster network router

> **TIP**
>
> **Correct Answer: [b]** — Kernel SVM requires computing an $N \times N$ dense kernel matrix; oneDAL uses efficient block-kernel caching, vectorized matrix evaluations, and parallelized working set selection in the SMO solver
> **Technical Explanation:** Evaluating the RBF kernel $\exp(-\gamma||x_i - x_j||^2)$ across N samples requires massive memory and compute ($O(N^2)$). oneDAL partitions the kernel matrix into cache-conscious tiles, executes vectorized SIMD kernel calculations, and parallelizes the Sequential Minimal Optimization (SMO) working set updates across all CPU cores.

Question 48 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Hard | Heterogeneous oneAPI Acceleration]

Topic: Targeting Intel GPUs with oneDAL / `sklearnex`
Question: How can an engineer direct an Intel-accelerated Scikit-learn estimator (e.g., `KMeans`) to execute on an Intel Data Center GPU Max rather than the host Xeon CPU?

- **a)** By using the `dpctl` device context: `with dpctl.device_context('gpu'): kmeans.fit(X)`
- **b)** By setting the environment variable `USE_GPU=1`
- **c)** By renaming the Python script to `gpu_script.py`
- **d)** By disconnecting the CPU power supply

> **TIP**
> **Correct Answer: [a]** — By using the `dpctl` device context: `with dpctl.device_context('gpu'): kmeans.fit(X)`
> **Technical Explanation:** Intel Extension for Scikit-learn integrates with `dpctl` (Data Parallel Control). Wrapping execution in `with dpctl.device_context('gpu'):` (or passing SYCL queues/USM tensors) instructs oneDAL to offload the compute kernels directly to the specified Intel GPU device.

Question 49 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Medium | Algorithmic Acceleration]

Topic: Principal Component Analysis (PCA) SVD Acceleration
Question: In high-dimensional feature spaces ($100,000 \times 1,000$), which core mathematical operation in PCA is accelerated by Intel oneDAL?

- **a)** Singular Value Decomposition (SVD) and symmetric eigenvalue decomposition of the covariance matrix ($X^T X$)
- **b)** Sorting rows alphabetically by feature name
- **c)** Random Gaussian noise injection
- **d)** Huffman coding compression

> **TIP**
> **Correct Answer: [a]** — Singular Value Decomposition (SVD) and symmetric eigenvalue decomposition of the covariance matrix ($X^T X$)
> **Technical Explanation:** PCA relies on computing either the covariance matrix and its eigendecomposition or the direct Singular Value Decomposition (SVD) of the centered data matrix ($X = U\Sigma V^T$). oneDAL optimizes this using vectorized BLAS GEMM ($X^T X$) and parallel divide-and-conquer SVD algorithms in oneMKL.

Question 50 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration | Easy | Verification & Toolchains]

Topic: Scikit-Learn Patch Verification
Question: How can a developer programmatically verify inside Python that Scikit-Learn has been successfully patched by `scikit-learn-intelex`?

- **a)** By printing `sklearn.cluster.KMeans.__module__` and checking that it points to `daal4py.sklearn` or `sklearnex`
- **b)** By checking if the computer's cooling fan is running
- **c)** By verifying that the script executes in exactly 0.000 seconds

- **d)** By checking the operating system version string

> **TIP**
>
> **Correct Answer: [a]** — By printing `sklearn.cluster.KMeans.__module__` and checking that it points to `daal4py.sklearn` or `sklearnex`
>
> **Technical Explanation:** When `patch_sklearn()` runs, it modifies Scikit-learn's class references. Inspecting `sklearn.cluster.KMeans.__module__` confirms the replacement, returning a path inside `daal4py.sklearn.cluster` or `sklearnex` rather than stock `sklearn.cluster._kmeans`.
